API Security Testing Report
Week: Week 4
Module: API Security Testing
Environment: Kali Linux
Target Application: Damn Vulnerable Web Application (DVWA)

---

1. Introduction

API Security Testing focuses on identifying vulnerabilities in web applications by analyzing HTTP requests and responses between the client and the server. In this exercise, DVWA (Damn Vulnerable Web Application) was used as the target application to demonstrate how insecure input handling can lead to vulnerabilities such as SQL Injection.
Burp Suite was used as the primary tool to intercept and manipulate HTTP requests in order to test the application's security.

---

2. Objectives

The main objectives of this task were:

- Set up a vulnerable web application environment
- Capture HTTP traffic using Burp Suite
- Intercept and analyze requests
- Modify request parameters
- Identify potential SQL Injection vulnerabilities
- Document the testing process and findings

3. Tools Used

The following tools were used during the testing process:

- Kali Linux
- DVWA (Damn Vulnerable Web Application)
- Burp Suite Community Edition
- Apache Web Server
- MySQL Database

4. Testing Methodology

The API security testing process followed these steps:

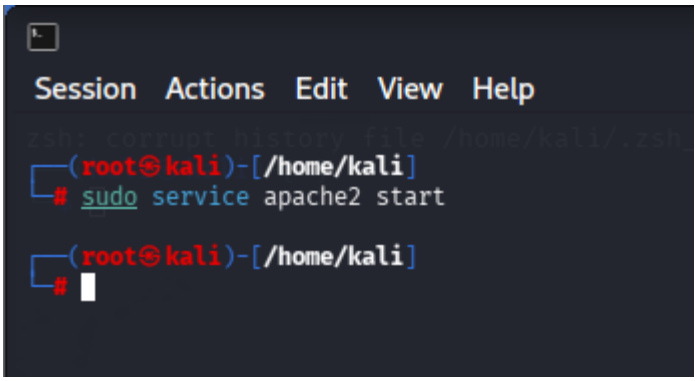
1. Start the Apache web server to host the DVWA application.
2. Start the MySQL database service required by DVWA.
3. Access the DVWA application through a web browser.
4. Launch Burp Suite to intercept HTTP traffic.
5. Enable the proxy intercept feature.
6. Capture requests from the DVWA application.
7. Forward intercepted requests to allow normal application behavior.
8. Send captured requests to Burp Suite Repeater.
9. Modify request parameters to test input validation.
10. Perform SQL Injection testing using crafted payloads.

5. Testing Steps and Evidence

Step 1 – Start Apache Server

The Apache web server was started to host the DVWA application locally.

Screenshot:



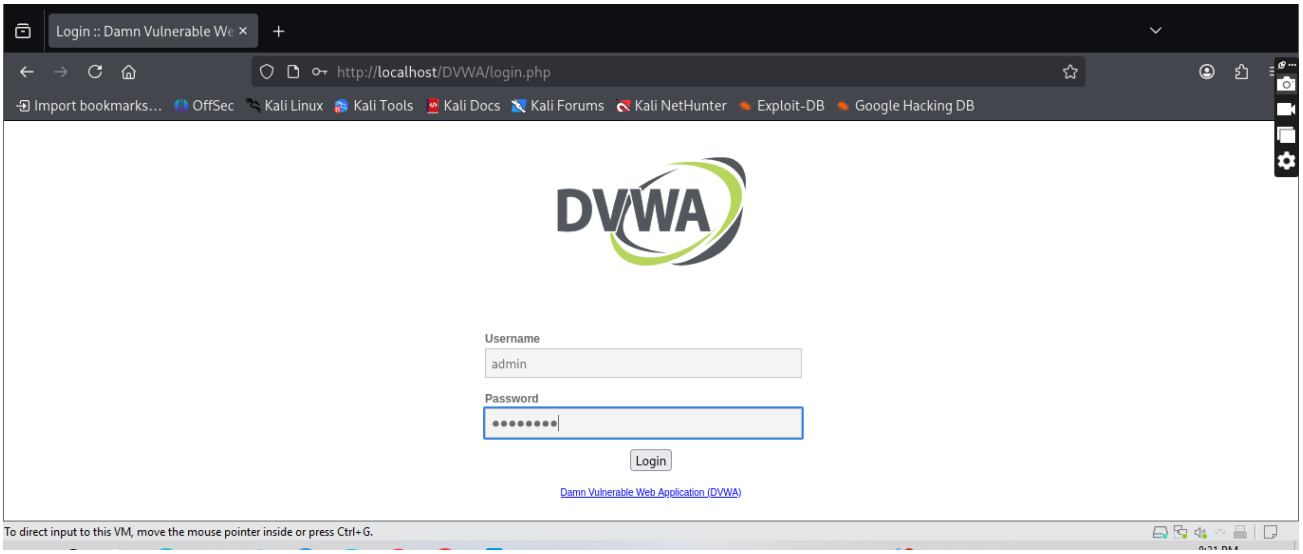
Step 2 – Access DVWA Dashboard

The DVWA application dashboard was accessed through the browser.

URL Used:

http://localhost/DVWA

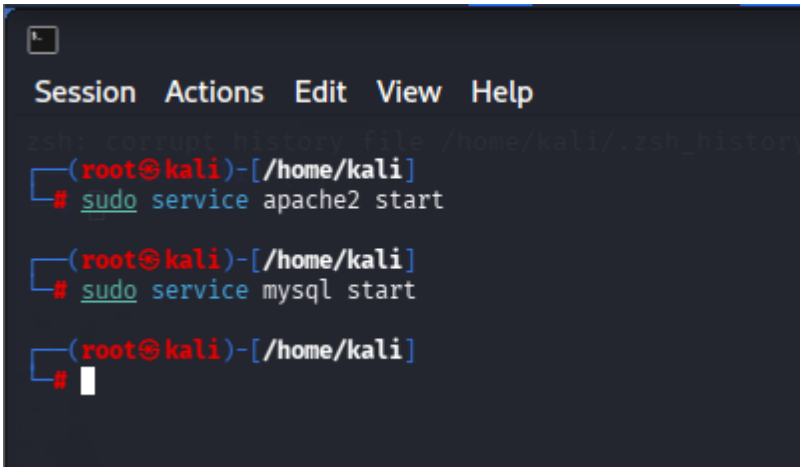
Screenshot:



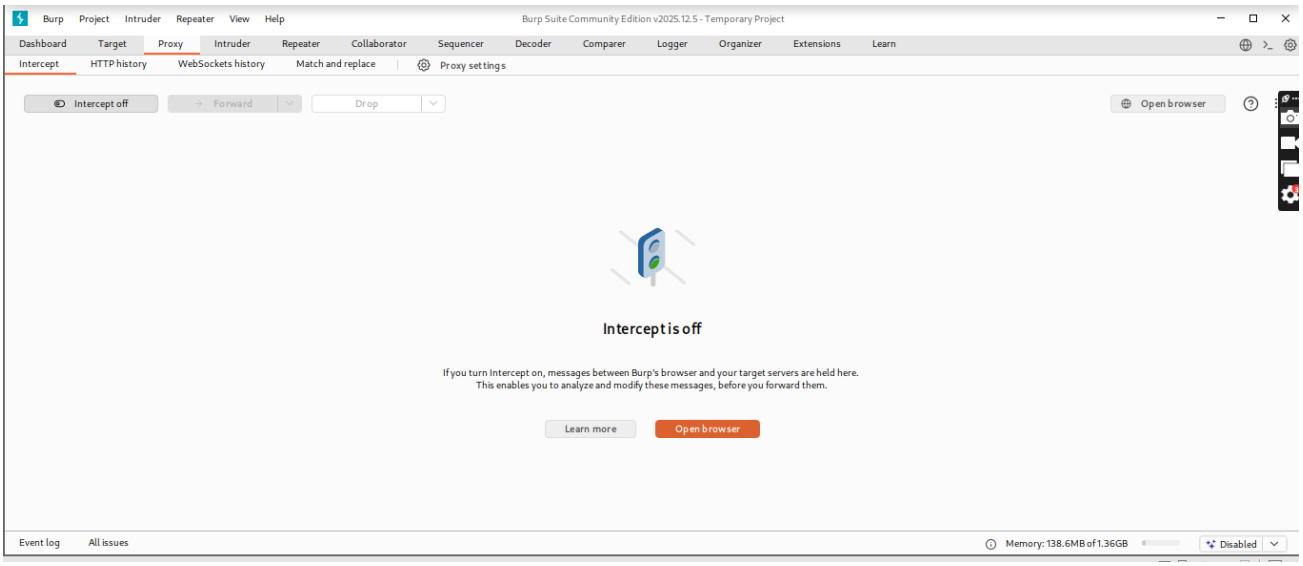
Step 3 – Start MySQL Service

The MySQL database service was started since DVWA relies on a backend database.

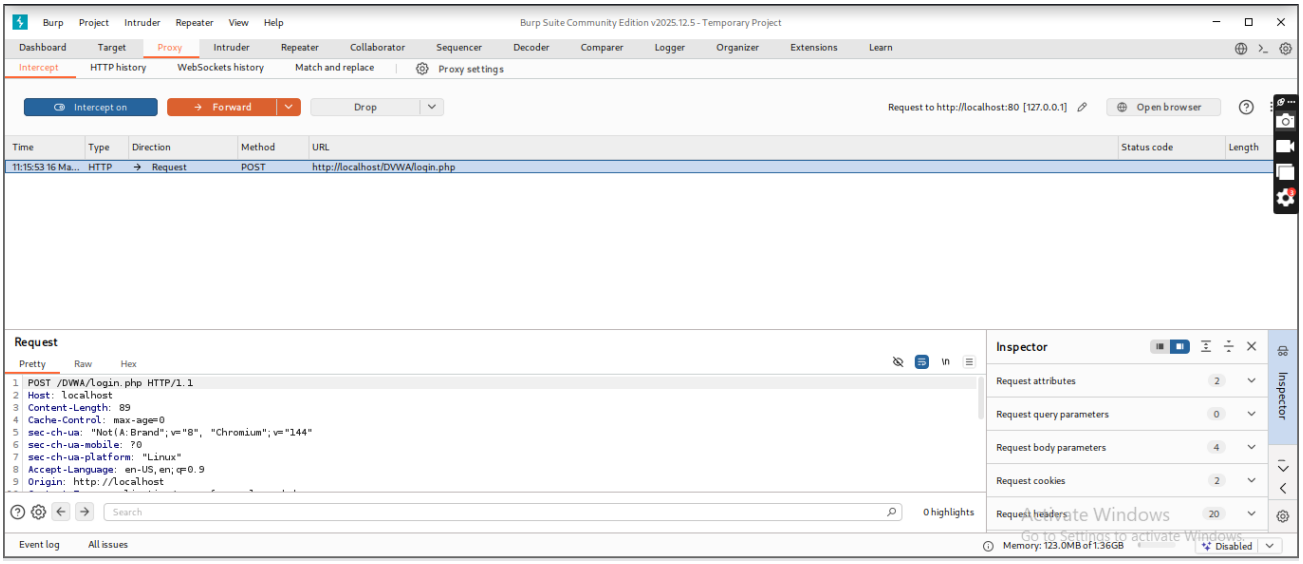
Screenshot:



Step 4 – Start Burp Suite
Burp Suite was launched to intercept HTTP traffic between the browser and the DVWA application.
Screenshot:



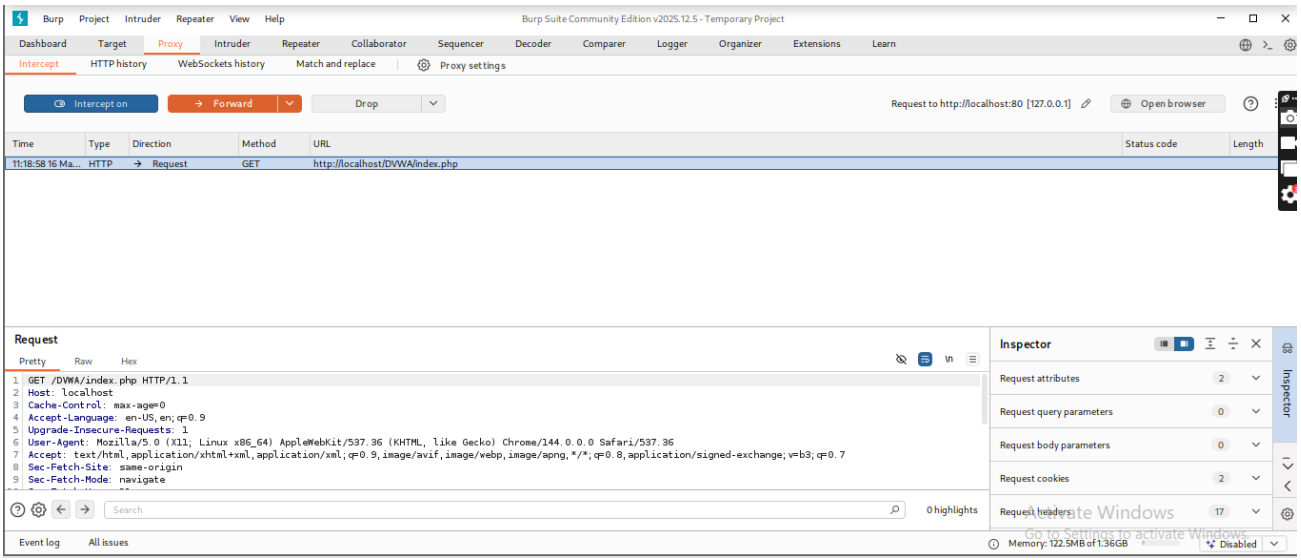
Step 5 – Intercept HTTP Request
Burp Proxy was enabled and a request from the DVWA application was intercepted.
Screenshot:



Step 6 – Forward the Request

The captured request was forwarded to the server to allow the application to continue processing.

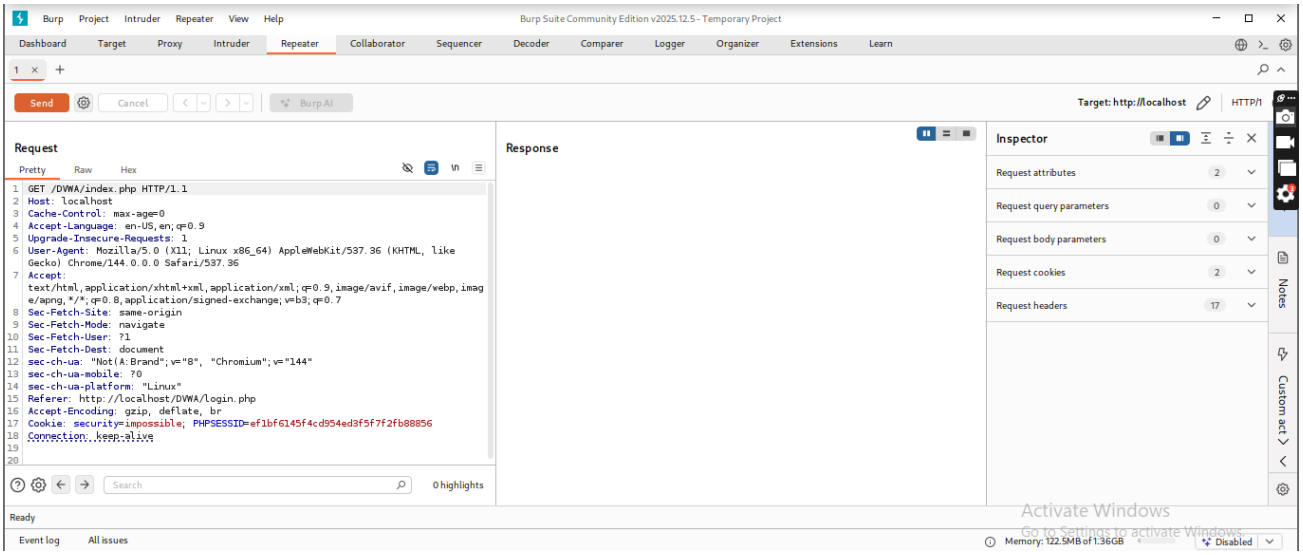
Screenshot:



Step 7 – Send Request to Repeater

The intercepted request was sent to Burp Suite's Repeater module for manual testing and modification.

Screenshot:



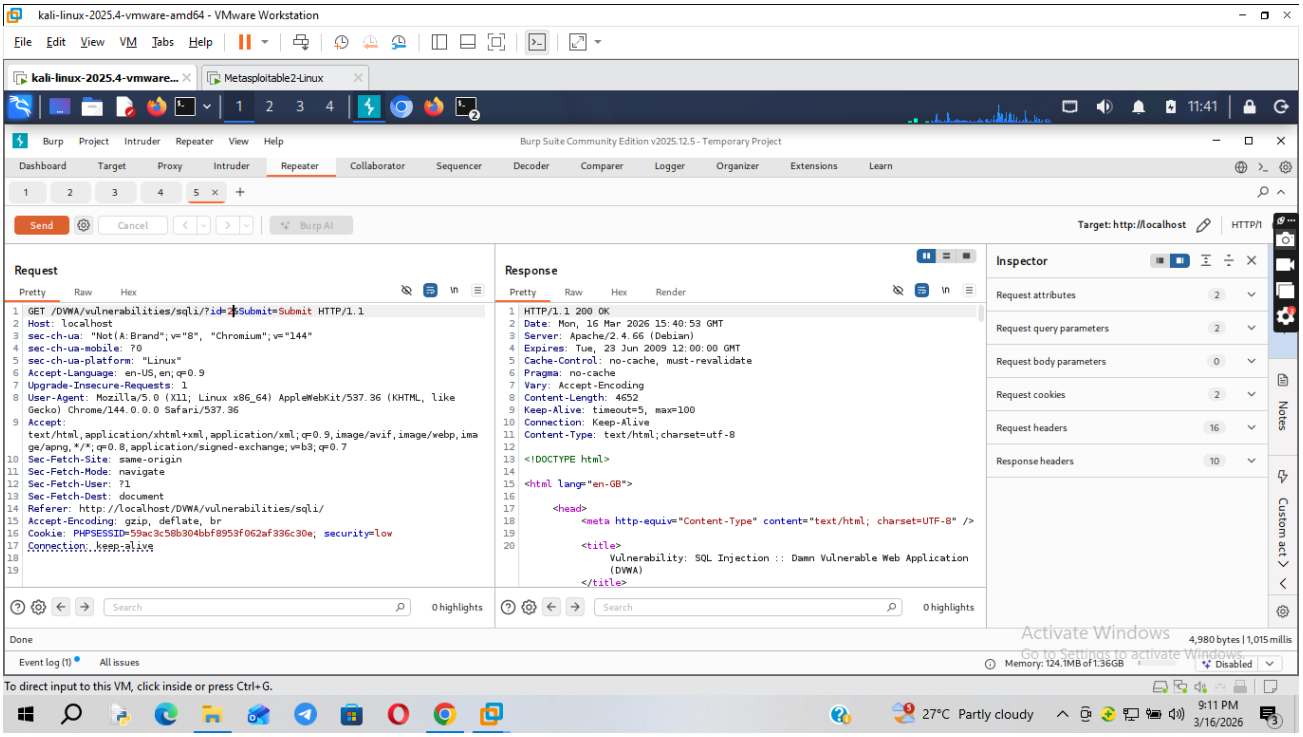
Step 8 – Modify Request Parameter

The parameter in the HTTP request was modified to test input validation.

Example modification:

id=1 → id=2

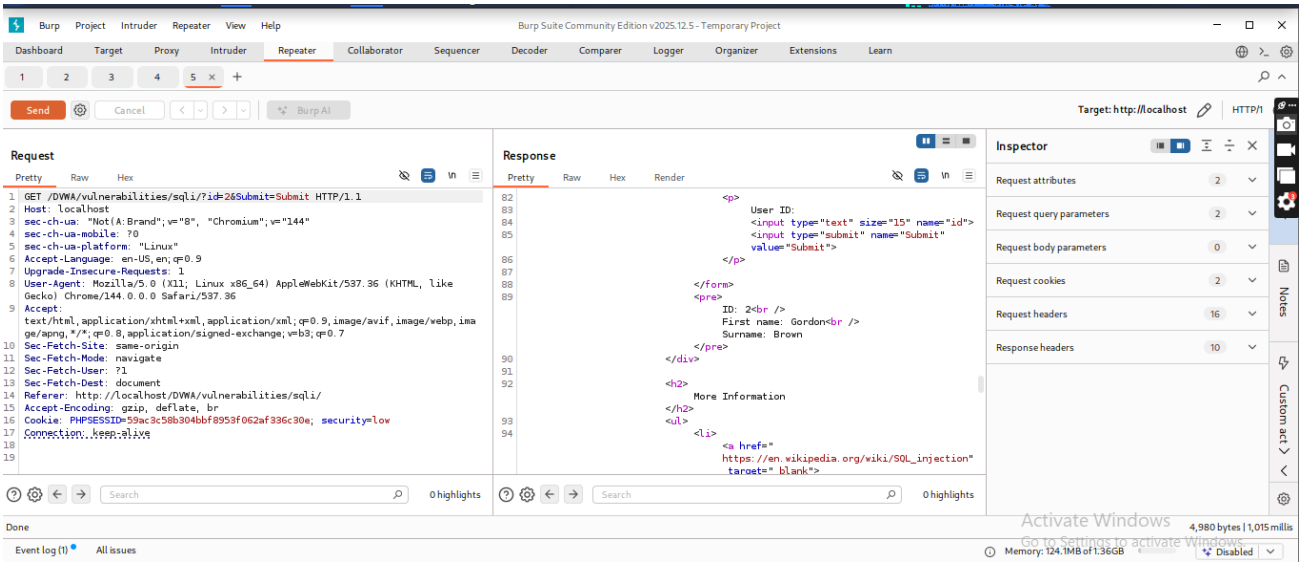
Screenshot:



Step 9 – Analyze Server Response

The server response was analyzed to observe how the application handled modified parameters.

Screenshot:



Step 10 – SQL Injection Testing

A SQL Injection payload was used to test the application's input validation.

Example payload used:

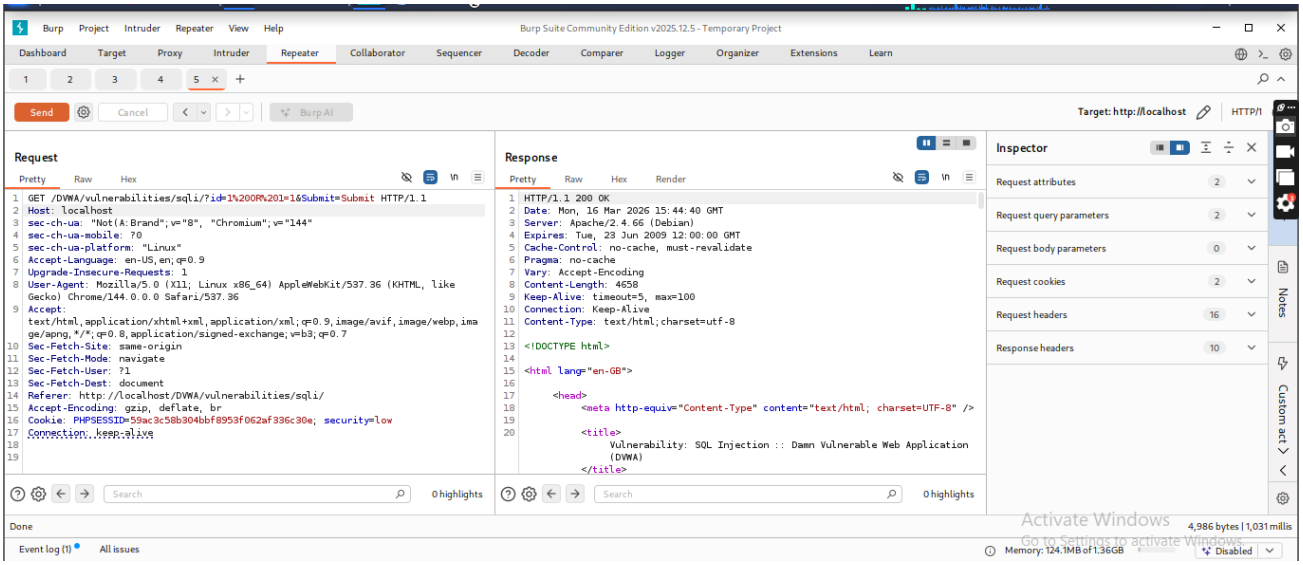
id=1 OR 1=1

Encoded payload:

id=1%20OR%201=1

The response indicated that the application returned multiple database records, confirming the presence of a SQL Injection vulnerability.

Screenshot:



6. Vulnerability Identified

SQL Injection
Description
SQL Injection is a vulnerability that occurs when user input is directly included in SQL queries without proper validation or sanitization.
Impact
If exploited successfully, attackers may:

- Access sensitive database information
- Bypass authentication mechanisms
- Modify or delete database records
- Gain unauthorized administrative access

Risk Level
High

---

7. Recommendations

To prevent SQL Injection vulnerabilities, the following security measures should be implemented:

- Use parameterized queries or prepared statements
- Validate and sanitize all user inputs
- Implement proper authentication and authorization mechanisms
- Use Web Application Firewalls (WAF)
- Perform regular security testing and vulnerability assessments

---

8. Conclusion

The API security testing exercise successfully demonstrated how improper input validation can lead to SQL Injection vulnerabilities. By intercepting and modifying HTTP requests using Burp Suite, it was possible to manipulate parameters and observe how the application processed malicious inputs. Implementing secure coding practices and proper input validation mechanisms is essential to protect applications from such attacks.